

Week 11: Function, User Defined Data Type & Macro

General Instructions:

1. Remove existing directories and create a fresh directory of your own. Write all the programs inside the newly created directory.
 2. Remove the directory that you have created at the end of your Lab session.
 3. Inform one of the TAs before you leave the Lab.
-

1. Function

Say, you are asked to write a C program to add factorial of two positive integers (say, $a!$ + $b!$). A naïve way of solving this problem is to write program codes to estimate factorial for both the number and perform addition. This approach can be illustrated below.

```
#include <stdio.h>

main(){
    int a, b,c;
    int i, fac_a=1, fac_b=1;

    //Read the two integers
    scanf("%d %d",&a, &b);

    //estimate factorial of a
    for(i=1;i<=a;i++)
        fac_a=fac_a*i;

    //estimate factorial of b
    for(i=1;i<=b;i++)
        fac_b=fac_b*i;

    c=fac_a+fac_b;
    printf("%d",c);
}
```

In the above example, you will notice that same program section for estimating factorial is repeated twice. As it repeats only twice, in this example, you may feel natural. What if, you have to repeat it say ten times, or more times? What if you have to repeat a large block of code segment? In the above example, we repeat only a code segment consisting of only two lines. It could be a large code segment. In such a scenario, several issues may be arisen.

- Boredom: You may get bore repeating the same/similar set of codes many times. As a result, you may be carelessly repeating the subsequent repetitions, and make mistakes.
- The size of program will be large. As a result, you may face difficulty in resolving errors.
- Difficult to understand the code or debug
- If the repeating codes performs a complex operation, you may feel like taking help from others. Just imagine if you yourself have to write program for *printf* or *scanf* functions.

It is always good to divide your program into logically/conceptually different tasks, and write separate function for each conceptually separate task.

What is a function?

A function is a program segment which performs a well-defined task.

Why we need functions?

- To improve the readability of code.
- Improves the reusability of the code, same function can be used in any program rather than writing the same code from scratch.
- Debugging of the code would be easier if you use functions, as errors are easy to be traced.
- Reduces the size of the code, duplicate set of statements are replaced by function calls.
- Allows one to develop a program in a modular fashion (divide-and-conquer)

Two types of functions:

- **Standard predefined function:** It comes with C library. The function like printf(), scanf() are standard predefined function.
- **User Defined Function:** Programmer defines his/her own functions to perform a specific task.

Note that **main()** is also a function, which defines the starting point of execution. C program always starts execution from main function.

Syntax of Function:

One need to know the following three terminologies. Let us consider function to estimate factorial to illustrate these terminologies.

1. **Function declaration:** It defines the syntax of a function that has to be followed while writing the body of the function, or using the function. It consists of three parts.
 - a. Name of the function
 - b. Data type of the return value, if any
 - c. Data types of the parameters, if any. The order of the parameters.

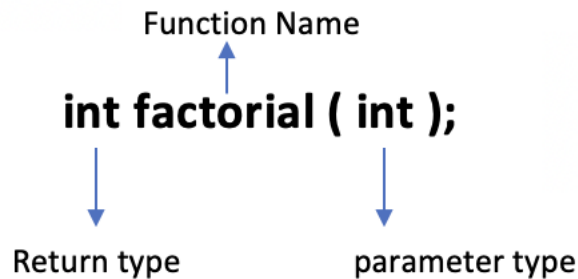
Syntax:

<Return Data type> Name-of-the-function (<list of data type of the parameters>);

Let us consider a function declaration for a factorial function.

int factorial (int);

The above declaration says, ***factorial*** is the name of the function. This function takes an integer value, and returns an integer value. While writing the body of the function or while using the function, the above syntax should be maintained.



Note that parameter name is optional. Only the parameter data type is required in the declaration.

It is not necessary that every function should return a value. It can just perform a task without returning any value. For example, it just calculates the factorial and just print inside the function. In such a case, we use *void* in place of the return data type.

void factorial (int);

Similarly, a function may not take any parameter. Say, we just print “Hello World”. We use *void* as parameter.

void factorial (void);

2. **Function definition:** It defines the body of the function. The syntax of a function definition is as follows.

<Return Data type> Name-of-the-function (<list of parameters with their data types and name>){

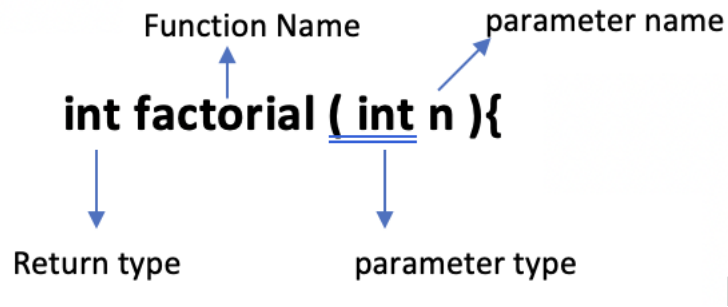
Set of statements - Code segment

}

For the factorial function, its definition can be defined as follows.

```
int factorial ( int n ){
    int i, fac=1;
    for(i=1;i<=n;i++)
        fac=fac*i;
    return fac;
}
```

Unlike declaration, in the definition, you should also give the name of the parameters along with their data types. The ***return fac;*** returns the value of fac variable and exits from the function. If a function has a return data type, it should always have a return statement.



As mention above, a function may not return a value. Say, estimate and just print factorial value.

```
void factorial ( int n ){  
    int i, fac=1;  
    for(i=1;i<=n;i++)  
        fac=fac*i;  
    printf("Factorial of %d is %d", n, fac);  
}
```

The above function does not return any value.

Similarly, a function may not return any value and take any parameter.

```
void printHelloWorld ( void ){  
    printf("Hello World");  
}
```

A function may take more than one parameter. The following function takes two integer values and return summation of the two numbers. **The parameters are separated by comma (,).**

```
int add (int a, int b){  
    return a+b;  
}
```

A function may take parameters of different data types.

```
int addOrSub (int a, int b, char opt){  
    if (opt=='+')  
        return a + b;  
    else if (opt=='-')  
        return a - b;  
}
```

3. **Function Call:** You call a function from another function to perform a task. While calling a function, you should maintain the syntax defined in its declaration. The below example calls **factorial** function from main function.

```
main(){
    int a, b,c;

    //Read the two integers
    scanf("%d %d",&a, &b);

    c = factorial(a) + factorial(b);
    printf("%d",c);
}
```

Putting all together, we can write the factorial function as follows.

```
#include <stdio.h>

// Function declaration
int factorial( int );

main(){
    int a, b,c;

    //Read the two integers
    scanf("%d %d",&a, &b);

    // Function call
    c = factorial(a) + factorial(b);
    printf("%d",c);
}

// Function definition
int factorial ( int n ){
    int i, fac=1;
    for(i=1;i<=n;i++)
        fac=fac*i;
    return fac;
}
```

Example 1: Write a calculator program which can perform addition, subtraction, division, and multiplication of two numbers.

```

#include <stdio.h>

// Function declaration
int add ( int, int );
int sub ( int, int );
int mul ( int, int );
float div ( int, int );

main(){
    int a, b;
    char opt;
    //Read the two integers
    printf("Enter the two numbers: ");
    scanf("%d %d",&a, &b);

    printf("Enter the option: ");
    scanf("%c", &opt);
    while(opt=='+' || opt=='-' || opt=='/' || opt=='*'){
        if(opt=='+')
            printf("Sum of %d and %d is : %d", a, b, add(a,b));
        else if(opt=='-')
            printf("Subtraction of %d and %d is : %d", a, b, sub(a,b));
        else if(opt=='*')
            printf("Multiplication of %d and %d is : %d", a, b, mul(a,b));
        else if(opt=='/')
            printf("Division of %d and %d is : %f", a, b, div(a,b));
    }
}

// Function definition
int add ( int a, int b ){
    return a+b;
}

// Function definition
int sub ( int a, int b ){
    return a-b;
}

// Function definition
int mul ( int a, int b ){
    return a*b;
}

// Function definition
float add ( int a, int b ){
    return a/b;
}

```

Compiler scan the program from top to bottom. If you put the function definition before the main function, then the explicit function declaration is not required.

```
#include <stdio.h>

// Function definition
int add ( int a, int b ){
    return a+b;
}

// Function definition
int sub ( int a, int b ){
    return a-b;
}

// Function definition
int mul ( int a, int b ){
    return a*b;
}

// Function definition
float div ( int a, int b ){
    return a/b;
}

main(){
    int a, b;
    char opt;
    //Read the two integers
    printf("Enter the two numbers: ");
    scanf("%d %d",&a, &b);

    printf("Enter the option: ");
    scanf("%c", &opt);

    while(opt=='+' || opt=='-' || opt=='/' || opt=='*'){
        if(opt=='+')
            printf("Sum of %d and %d is : %d", a, b, add(a,b));
        else if(opt=='-')
            printf("Subtraction of %d and %d is : %d", a, b, sub(a,b));
        else if(opt=='*')
            printf("Multiplication of %d and %d is : %d", a, b, mul(a,b));
        else if(opt=='/')
            printf("Division of %d and %d is : %f", a, b, div(a,b));
    }
}
```

Example 2: Given an array of integers, find the minimum element in the array.

/* In this example, you will learn how to pass array as parameter, and call one user defined function from another user defined function. */

```
#include <stdio.h>

// Find minimum between two numbers.
int min(int a, int b){
    return (a > b) ? b : a;
}

int getMin(int a[], int b){
    int m = a[0];
    for (int i = 1; i < b; i++)
        res = min(m, a[i]);
    return res;
}

main(){
    int a[] = { 10, 5, 4, 6, 2 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("The minimum element the array is: %d \n",getMin(arr, n));
}
```

Note that the variable a and b are used repeatedly in different function. It is allowed because the variables declared in each of the function are local to its respective function. Therefore, there is no conflict among the scopes of the respective variables.

More on function will be discussed in next tutorial.

2. User Defined Data Type

The data types that we have used so far such as int, char, float, etc. are primitive data types which come along with the compiler. We may wish to define our own data type. Such a data type is known as user defined data type. C supports the four types of user defined data types.

1. Structure
2. Union
3. Enum
4. Typedef

2.1 Structure: Structures are used to group items of different types into a single type. The “struct” keyword is used to define a structure. The size of the structure is defined by the total size of all of its members.

Syntax:

```
struct structure_name {  
    data_type member_name1;  
    data_type member_name1;  
    ....  
    ....  
};
```

Let us say, we want to define a complex number with its real and imaginary members. We can define it as follows. The name of the data type that we just define is **complex**.

```
struct complex{  
    float real;  
    float imag;  
};
```

An instance/variable of a structure is declared as follows.

```
struct complex a, b;
```

In the above example, the a and b are two variables of user defined data type complex. For accessing the members of the variable, we use dot (.) operator.

Let us write a program to create complex numbers and perform arithmetic operation on complex numbers.

```
#include <stdio.h>  
  
// Define the structure "Complex"  
struct complex {  
    float real;  
    float imag;  
};
```

```

main() {
    // Declare variables
    struct complex a, b, c;

    // Read Inputs
    printf(" Entre the first Complex Number (real imag): ");
    scanf("%f %f", &a.real, &a.imag);

    printf("Entre the second Complex Number (real imag): ");
    scanf("%f %f", &b.real, &b.imag);

    // Add the two complex numbers
    c = add(a,b);

    // Display the results
    printf("\nSum of Complex Numbers is:\n");
    display(c);

    // Multiply the two complex numbers
    c = mult(a, b);

    // Display the results
    printf("\nProduct of two Complex Numbers is:\n");
    display(c);
}

// Function to add two complex numbers
struct complex add(struct complex x, struct complex y) {
    struct complex z;
    z.real = x.real + y.real;
    z.imag = x.imag + y.imag;
    return y;
}

// Function to multiply two complex numbers
struct complex mult(struct complex x, struct complex y) {
    struct complex z;
    z.real = (x.real * y.real) - (x.imag * y.imag);
    z.imag = (x.real * y.imag) + (x.imag * y.real);
    return z;
}

```

```
// Function to display a complex number
void displayComplex(struct complex a) {
    printf("%.2f + %.2fi\n", a.real, a.imag);
}
```

Similarly, we can define a structure defining student record.

```
struct student{
    char name[50];
    char roll[10];
    int age;
    float cpi;
};
```

As name is a string, i.e., array of characters, we will see example in the next tutorial, where we learn string handling. In the similar manner, we can define any structure consisting of multiple data members.

2.2 Union: Unlike struct, union behaves as **one of the data member**. That means, a union variable stores one of its members. What makes a union different is that all the members in the union are stored in the same memory location resulting in only one member containing data at the same time. The size of the union is the size of its largest member. Union is declared using the “**union**” keyword.

Syntax:

```
union union_name {
    datatype member1;
    datatype member2;
    ...
};
```

Example:

```
#include <stdio.h>

// Declaring a union. It can store either an integer value or float, but not both
union data {
    int i;
    float f;
};

int main()
{
    // Create a variable of data union type
    union data a;
    a.i = 10;
```

```

printf("the value is: %d\n", a.i);

// Overwrite the earlier value
a.f = 3.14;
printf("the value is: %f\n", a.f);

//a.i is no more valid, after the above assignment. Check the following printf
printf("the value is: %d\n", a.i);

}

```

2.3: Enumeration

Enum is short for “Enumeration”. It allows the user to create custom data types with a set of named integer constants. The “enum” keyword is used to declare an enumeration.

Syntax:

```
enum enum_name {const1, const2, ..., constN};
```

Example:

```
enum series{ first, second, third, four};
```

The member first will have value 0, second will have 1, third 2 and four 3.

```

#include<stdio.h>

//declaration
enum series { first, second, third, four};

main()
{
    enum series no =third;
    printf("No is %d\n", no);
}

```

Example 2:

```

#include<stdio.h>
enum rating { LOW, MEDIUM, HIGH };

main() {
    // Create an enum variable and assign rating
    enum rating myRating = MEDIUM;

    // Print the rating
}

```

```
    printf("%d", myRating);  
}
```

You can also assign explicit value to the members.

Syntax:

```
enum enum_name {  
    const1 = 8;  
    const2 = 4;  
}
```

Example:

```
#include<stdio.h>  
enum rating { LOW = 0, MEDIUM = 5, HIGH = 10 };  
  
main() {  
    // Create an enum variable and assign rating  
    enum rating myRating = MEDIUM;  
  
    // Print the rating  
    printf("%d", myRating);  
}
```

2.4 Typedef: You are bore with the existing name of the data types. I want to use my favorite names. You can do it with **typedef**.

Syntax:

```
typedef existing_name alias_name;
```

Example: typedef int integer

Now, you can use **integer** in place of **int**.

```
#include <stdio.h>  
  
typedef int integer;  
  
int main()  
{  
    integer a = 10;  
    printf("%d \n", a);  
    return 0;  
}
```

3 Macros

A **macro** is a piece of code in a program that is replaced by the value of the macro. Macro is defined by **#define** directive. Whenever a macro name is encountered by the compiler, it replaces the name with the definition of the macro. **Macro definitions need not be terminated by a semi-colon(;;).**

Example:

```
#include <stdio.h>

// Macro definition. Wherever MAX, appear, it will be replaced by its value 10 by
// compiler

#define MAX 10
int main() {
    int i;

    for(i=0; i<MAX; i++){
        printf("%d ", i);
    }
    return 0;
}
```

Example 2: Find area of a rectangle.

```
#include <stdio.h>

// The Macro AREA(l, b) will be replace by (l * b)
#define AREA(l, b) (l * b)

// Driver Code
int main()
{
    // Given lengths x and y
    int x = 4, y = 5, area;

    // Find the area using macros
    area = AREA(x, y);

    // Print the area
    printf("Area of rectangle is: %d", area);

    return 0;
}
```

A macro can also be defined in the form of a chain.

```
// C program to illustrate macros
#include <stdio.h>

// Macro definition
```

```

#define TWITTER FOLLOWERS
#define FOLLOWERS 1000

// Driver Code
int main()
{
    // Print the message
    printf("I have %d followers on X",TWITTER);

    return 0;
}

```

In the above example, TWITTER is replaced by FOLLOWERS and FOLLOWERS is replace by 1000.

A macro can have multiple lines.

```

#include <stdio.h>

// Multi-line Macro definition
#define ELE 1, \
           2, \
           3

int main()
{
    // Array arr[] with elements
    // defined in macros
    int arr[] = { ELE };

    // Print elements
    printf("Elements of Array are:\n");

    for (int i = 0; i < 3; i++) {
        printf("%d  ", arr[i]);
    }
    return 0;
}

```

The above example defines the elements of an array using macro.

A macro can also be used to define a function.

Example:

```

#include <stdio.h>

#define min(a, b) ((a) < (b)) ? (a) : (b)

```

```
int main(){
    int a = 6;
    int b = 3;

    printf("Minimum of %d and %d is %d\n", a, b, min(a,
b));

    return 0;
}
```

Example: Area of circle

```
#include <stdio.h>
#define PI 3.1416
#define AREA(r) (PI*(r)*(r))

int main() {
    float a = 7;    // radius of circle

    printf("Area is %f", AREA(a));
    return 0;
}
```